Business Analytics

Group Assignment: Digital Ads Clustering

Submitted By -

NAME	Roll Number
NAMAN JAIN	FT272022
OJASVI GAUR	FT261010R
HARSHRAJ	FT272017
TANMAY	FT272069
PUSHKAR	FT272056
TANYA	FT272073

Background and objective

ads24x7 is a digital marketing company that has just raised $10 million in seed funding and wants to build out a marketing analytics practice. The marketing intelligence team handed us 25,857 ad records and asked a fairly open question - What kinds of ads are we actually running?

The brief was to group the ads into homogeneous segments using clustering. We built the segmentation on the 3 cost and engagement metrics that digital marketers use every day, namely click through rate, cost per mille and cost per click. Everything below follows from that work. The Python code sits in the attached file and this document carries the interpretation.

What the data looked like before we touched it

The file has 25,857 rows and 19 columns. There were no duplicate records, so nothing had to be dropped on that count. Three columns had gaps: CTR and CPM were each missing 6,465 values, and CPC was missing 7,527.

Those gaps turned out to be recoverable. All 3 are derived metrics, and the raw ingredients such as Clicks, Impressions and Spend were fully populated. So rather than filling them with a mean or a median, we recalculated them from the formulas given in the brief.

Step 2 : Reading the data and basic analysis

```
df = pd.read_excel('Clustering Clean Ads_Data.xlsx')
df.head()
```

	Timestamp	InventoryType	Ad - Length	Ad- Width	Ad Size	Ad Type	Platform	Device Type	Format	Available_Impressions	Matched_Queries	Impressions	Clicks	Spend	Fee	Revenue	CTR	CPM	CPC
0	2020-9-2-17	Format1	300	250	75000	Inter222	Video	Desktop	Display	1806	325	323	1	0.0	0.35	0.0	0.0031	0.0	0.0
1	2020-9-2-18	Format1	300	250	75000	Inter223	Web	Mobile	Display	1979	384	380	0	0.0	0.35	0.0	0.0000	0.0	NaN
2	2020-9-3-16	Format6	336	250	84000	Inter217	Web	Desktop	Video	1566	298	297	0	0.0	0.35	0.0	0.0000	0.0	NaN
3	2020-9-3-2	Format1	300	250	75000	Inter224	Web	Desktop	Display	643	103	102	0	0.0	0.35	0.0	0.0000	0.0	NaN
4	2020-9-3-13	Format1	300	250	75000	Inter225	Video	Mobile	Display	1550	347	345	0	0.0	0.35	0.0	0.0000	0.0	NaN

Number of rows : 25857

Number of columns : 19

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25857 entries, 0 to 25856
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Timestamp             25857 non-null  object
 1   InventoryType         25857 non-null  object
 2   Ad - Length           25857 non-null  int64
 3   Ad- Width             25857 non-null  int64
 4   Ad Size               25857 non-null  int64
 5   Ad Type               25857 non-null  object
 6   Platform              25857 non-null  object
 7   Device Type           25857 non-null  object
 8   Format                25857 non-null  object
 9   Available_Impressions 25857 non-null  int64
 10  Matched_Queries       25857 non-null  int64
 11  Impressions           25857 non-null  int64
 12  Clicks                25857 non-null  int64
 13  Spend                 25857 non-null  float64
 14  Fee                   25857 non-null  float64
 15  Revenue               25857 non-null  float64
 16  CTR                   19392 non-null  float64
 17  CPM                   19392 non-null  float64
 18  CPC                   18330 non-null  float64
dtypes: float64(6), int64(7), object(6)
memory usage: 3.7+ MB
```

```
# Checking the categorical variables present in the data
for col in ['InventoryType', 'Ad Type', 'Platform', 'Device Type', 'Format']:
    print(col, ':', df[col].nunique(), 'levels ->', list(df[col].unique()))
```

```
InventoryType : 7 levels -> ['Format1', 'Format6', 'Format5', 'Format7', 'Format4', 'Format3', 'Format2']
Ad Type : 14 levels -> ['Inter222', 'Inter223', 'Inter217', 'Inter224', 'Inter225', 'Inter226', 'Inter227', 'Inter228', 'Inter218', 'Inter219', 'inter230', 'Inter220', 'Inter221', 'Inter229']
Platform : 3 levels -> ['Video', 'Web', 'App']
Device Type : 2 levels -> ['Desktop', 'Mobile']
Format : 2 levels -> ['Display', 'Video']
```

A flagged issue in the CTR column

The assignment defines CTR as clicks divided by impressions, multiplied by 100. The values already sitting in the file do not follow that. One click on 323 impressions was stored as 0.0031, which is the plain ratio and not the percentage. If we had imputed the missing values using the formula as written and left the existing values alone, we would have ended up with one column holding two different units, and the clustering would have been built on nonsense.

```
df[['CTR', 'CPM', 'CPC']].describe().T
```

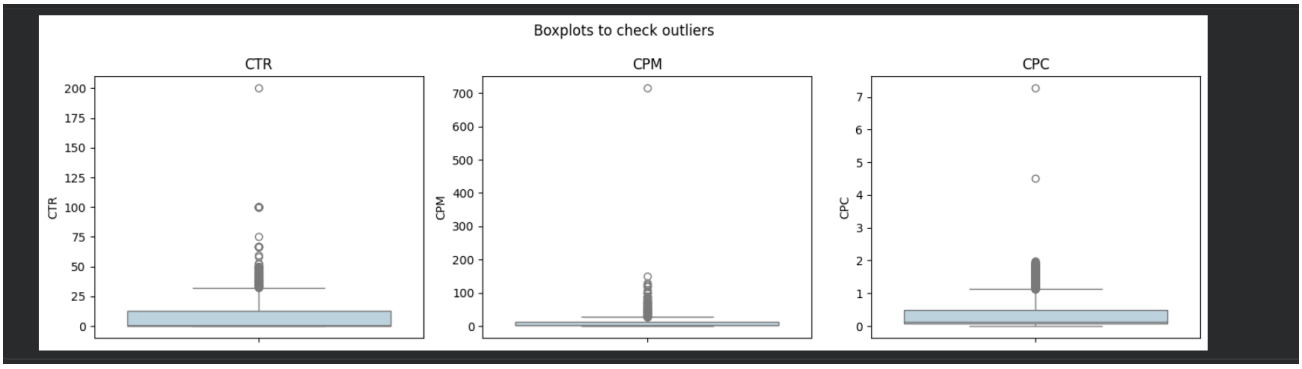
	count	mean	std	min	25%	50%	75%	max
CTR	25857.0	7.502185	9.128851	0.0	0.23	0.72	13.06	200.00
CPM	25857.0	7.524721	8.927584	0.0	1.57	2.96	12.45	715.00
CPC	25857.0	0.300301	0.338797	0.0	0.08	0.12	0.50	7.26

We handled this by multiplying the existing values by 100 as well, so the whole column is a genuine percentage. This does not change the clusters at all, because z-score scaling removes any constant multiplier, but it does mean the CTR figures quoted later in this document can be read as percentages without a footnote.

- 219 rows have zero impressions. The ad was booked but never actually served, so CTR and CPM for those rows are set to zero rather than left undefined
- 2,791 rows have zero clicks. The ad ran and nobody clicked. CPC is set to zero for these, since there is no click to attach a cost to
- One row has more clicks than impressions, which cannot happen in reality. It is a single record out of 25,857 so it does not move any result, but it is a data entry error and the marketing intelligence team should know about it

Outliers and why they were treated them

Using the standard interquartile range rule, CTR has about 1.2% outliers, CPM about 0.9% and CPC about 2.7%. All of them sit on the high side, which is what you would expect from advertising data. A few placements always cost far more than the rest.




```
# FINDING : CTR has around 1.2% outliers, CPM has around 0.9% and CPC has around 2.7%.
# The outliers are on the higher side in all three columns.

CTR : lower = -19.015 , upper = 32.305 , outliers = 302 ( 1.17 % )
CPM : lower = -14.75 , upper = 28.77 , outliers = 229 ( 0.89 % )
CPC : lower = -0.55 , upper = 1.13 , outliers = 709 ( 2.74 % )
```

Does K-Means need outlier treatment?

Yes, and the reason is built into how the algorithm works. K-Means puts every record into whichever cluster has the nearest centroid, and a centroid is just a mean. Means get dragged by extreme values. A few ads with a very high CPM can pull a centroid away from the group it is supposed to represent, and you end up with a cluster that has formed around noise instead of a real segment of the business.

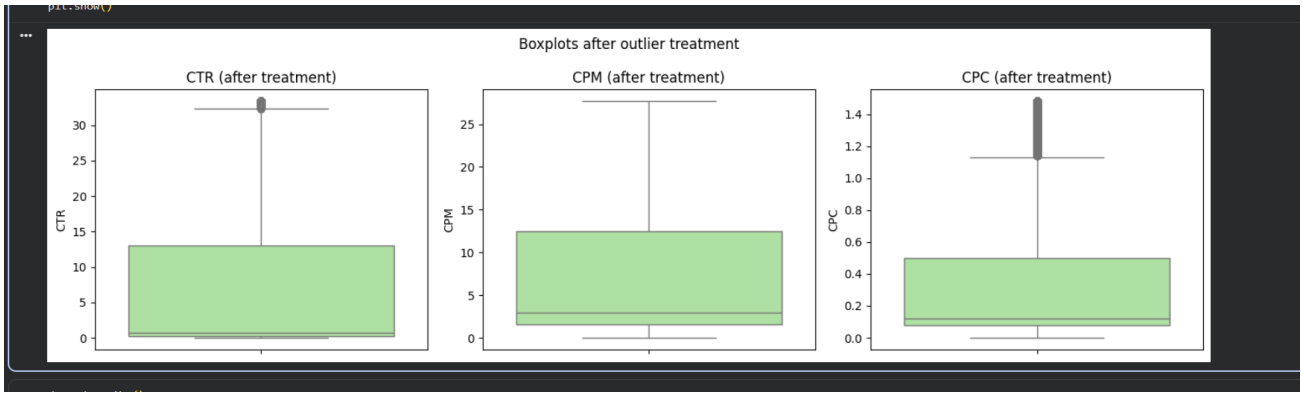
```
CTR capped at 33.333
CPM capped at 27.664
CPC capped at 1.48
```

Which method was used and why not delete it?

We capped the values at the 99th percentile instead of deleting the rows. The cutoffs worked out to 33.33 for CTR, 27.66 for CPM and 1.48 for CPC.

Deletion was tempting but wrong here. The high CPM records are not errors, they are the premium placements, and those are exactly the ads the business most wants to understand. Dropping them would have removed roughly 250 real ads from the final segmentation. Capping keeps all 25,857 records, keeps the ordering intact, and still stops the extremes from bending the centroids.

A different analyst could reasonably take a log transform instead and would not be wrong. The assignment allows for that, and the point is to make the choice deliberately rather than by default.




```
data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
CTR	25857.0	7.336181	8.204192	0.0	0.23	0.72	13.06	33.3333
CPM	25857.0	7.345780	6.899836	0.0	1.57	2.96	12.45	27.6644
CPC	25857.0	0.298408	0.329068	0.0	0.08	0.12	0.50	1.4800

Z-score scaling and its effect on speed

CTR is a percentage, CPM is a cost/thousand impressions and CPC is a cost/single click. Even after capping, CPM runs up to about 27 while CPC stops around 1.5. Without scaling, CPM would decide the distance between two ads almost single handedly, not because it matters more but because its numbers happen to be bigger.

Standardising each variable to a mean of zero and a standard deviation of one puts all three on equal footing.

```
data_scaled = pd.DataFrame(scale.fit_transform(data), columns=data.columns)
data_scaled.head()
```

	CTR	CPM	CPC
0	-0.856430	-1.064652	-0.906845
1	-0.894216	-1.064652	-0.906845
2	-0.894216	-1.064652	-0.906845
3	-0.894216	-1.064652	-0.906845
4	-0.894216	-1.064652	-0.906845

The speed argument

Scaling also makes the algorithm finish sooner. K-Means starts with random centroids and moves them until they stop shifting. On unscaled data the feature space is stretched out along whichever axis has the largest range, so the centroids have a long way to travel and need more passes before they settle. After scaling, everything sits in a compact and roughly symmetric cloud around the origin, the centroids start much closer to where they will end up, and convergence takes fewer iterations. On a file of 25,857 rows that is the difference between the model fitting instantly and the loop over ten values of K becoming something you wait for.

```
# Let us confirm that the mean is 0 and the standard deviation is 1 after scaling.
data_scaled.describe().T
```

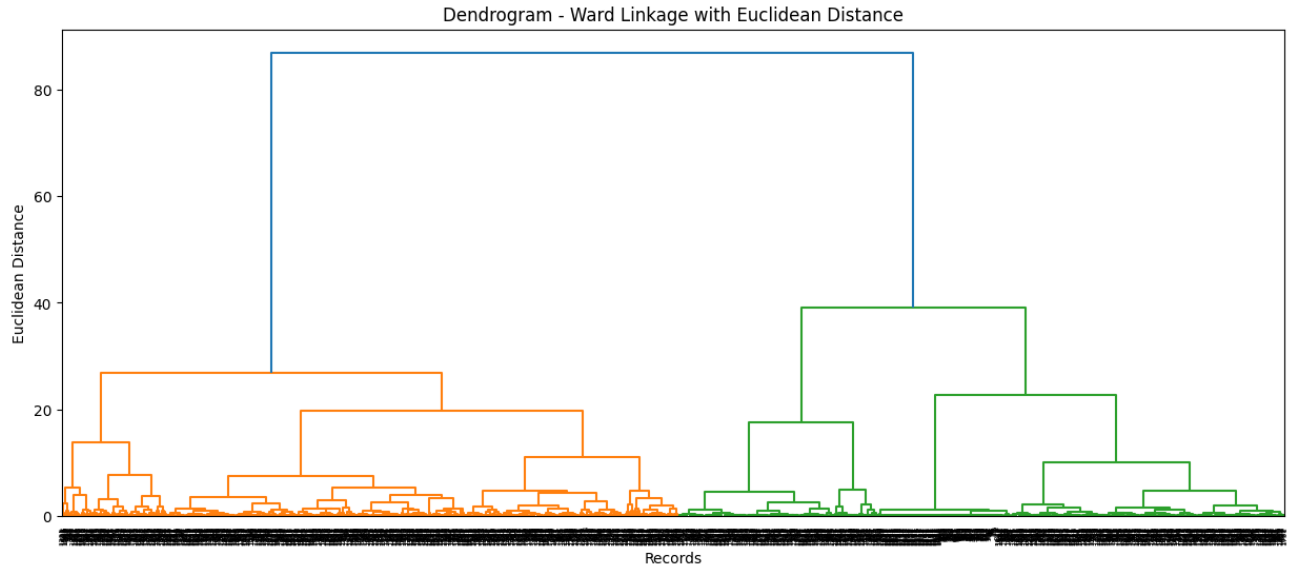
	count	mean	std	min	25%	50%	75%	max
CTR	25857.0	-8.793506e-17	1.000019	-0.894216	-0.866181	-0.806455	0.697684	3.168822
CPM	25857.0	1.582831e-16	1.000019	-1.064652	-0.837106	-0.635648	0.739774	2.944855
CPC	25857.0	7.474480e-17	1.000019	-0.906845	-0.663730	-0.542172	0.612628	3.590796

Choosing the number of clusters

Hierarchical clustering and the dendrogram

We built the dendrogram using Ward linkage with Euclidean distance, which is the pairing the brief asks for. Ward merges whichever two groups add the least to the total within cluster variance, so it tends to produce clusters of comparable size rather than one giant blob and a few strays.

Hierarchical clustering needs the distance between every pair of records, which on 25,857 rows comes to roughly 334 million pairs and several gigabytes of memory. It also produces a dendrogram so dense that you cannot read anything off it. We therefore drew it on a random sample of 2,000 records. Reading the longest uninterrupted vertical drops, the tree cuts naturally into 3 or 4 groups.




```
Cluster labels from hierarchical clustering : [1 2 3 4]
1    172
2    837
3    328
4    663
Name: count, dtype: int64
```

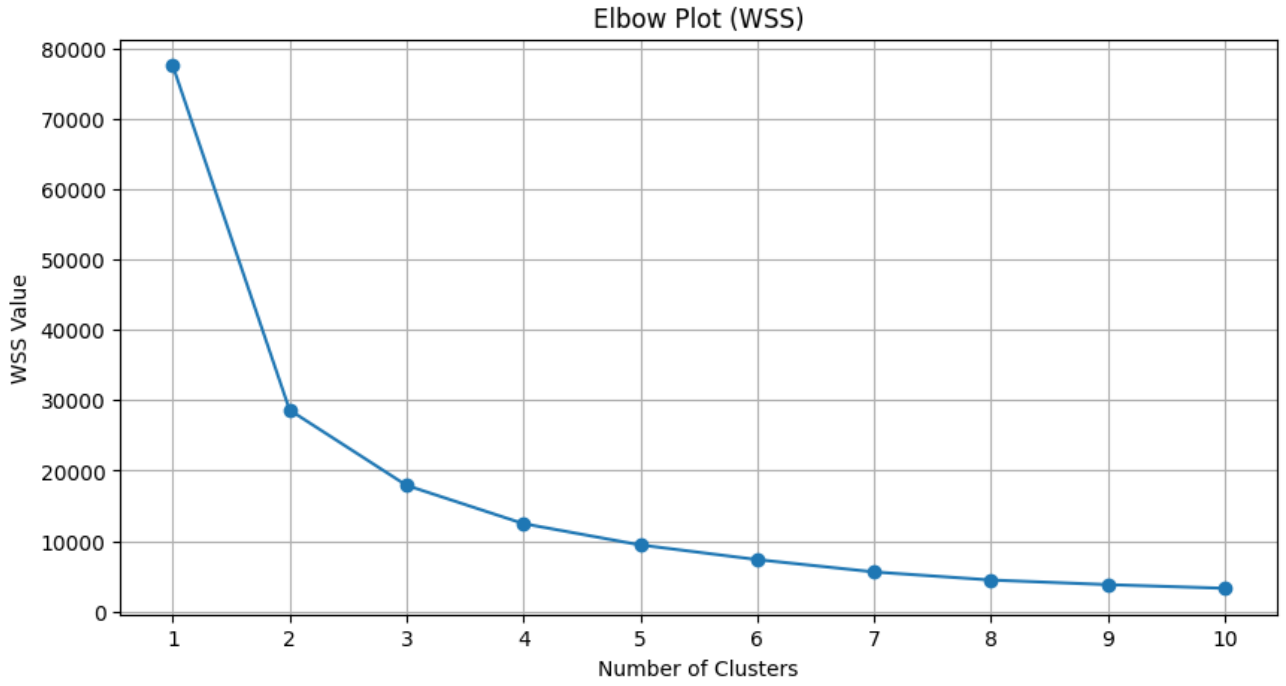
The elbow plot

The within cluster sum of squares falls steeply at first and then flattens. Reading the percentage drop at each step is easier than eyeballing the bend:

Step	WSS after	Reduction
1 to 2 clusters	28,610	63.1%
2 to 3 clusters	17,907	37.4%
3 to 4 clusters	12,465	30.4%
4 to 5 clusters	9,437	24.3%
5 to 6 clusters	7,355	22.1%

The gains stay meaningful up to 4 clusters and then the curve settles into a slow grind. The bend is at K = 4.

```
The WSS value for 1 clusters is 77571.0
The WSS value for 2 clusters is 28609.73
The WSS value for 3 clusters is 17906.8
The WSS value for 4 clusters is 12464.82
The WSS value for 5 clusters is 9436.78
The WSS value for 6 clusters is 7354.74
The WSS value for 7 clusters is 5601.83
The WSS value for 8 clusters is 4454.14
The WSS value for 9 clusters is 3797.3
The WSS value for 10 clusters is 3278.08
```




```
# FINDING : The drop is 63% from 1 to 2, 37% from 2 to 3 and 30% from 3 to 4. After
# 4 clusters the reduction falls to around 24% and keeps flattening. So the bend in
# the curve is around K = 4.
```

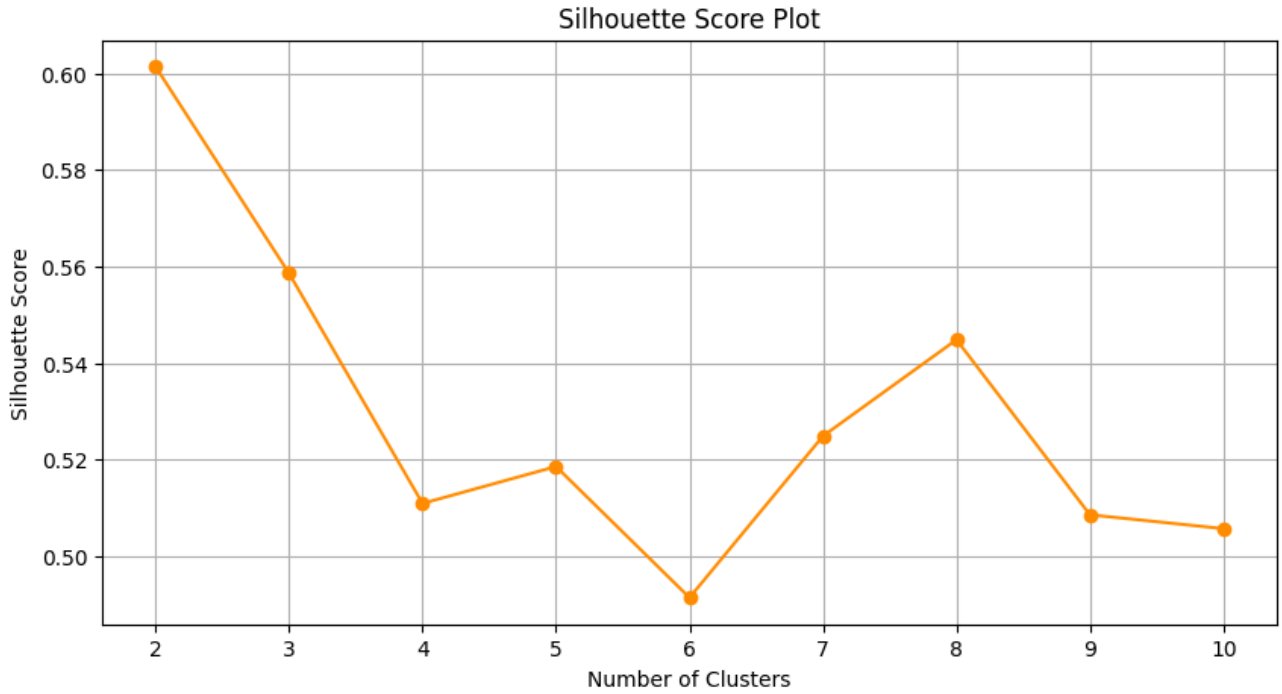
```
Going from 1 to 2 clusters reduces WSS by 63.1 %
Going from 2 to 3 clusters reduces WSS by 37.4 %
Going from 3 to 4 clusters reduces WSS by 30.4 %
Going from 4 to 5 clusters reduces WSS by 24.3 %
Going from 5 to 6 clusters reduces WSS by 22.1 %
Going from 6 to 7 clusters reduces WSS by 23.8 %
Going from 7 to 8 clusters reduces WSS by 20.5 %
Going from 8 to 9 clusters reduces WSS by 14.7 %
Going from 9 to 10 clusters reduces WSS by 13.7 %
```

Silhouette scores, and why we did not simply follow them

Clusters	Silhouette	Clusters	Silhouette
2	0.6014	7	0.5249
3	0.5586	8	0.5448
4	0.5109	9	0.5085
5	0.5185	10	0.5057
6	0.4914		

```python
# Printing the silhouette scores.
for i in range(2, 11):
    print('The silhouette score for', i, 'clusters is', round(sil_scores[i - 2], 4))
```

```
The silhouette score for 2 clusters is 0.6014
The silhouette score for 3 clusters is 0.5586
The silhouette score for 4 clusters is 0.5109
The silhouette score for 5 clusters is 0.5185
The silhouette score for 6 clusters is 0.4914
The silhouette score for 7 clusters is 0.5249
The silhouette score for 8 clusters is 0.5448
The silhouette score for 9 clusters is 0.5085
The silhouette score for 10 clusters is 0.5057
```



The highest score is at two clusters, and we are not taking it.

A 2-cluster solution splits the file into ads that got some traction and ads that got almost none. That is true, and it is useless. Nobody at ads24x7 can price inventory or plan a campaign off a binary flag. The silhouette score measures geometric separation, and geometric separation is not the same thing as commercial usefulness.

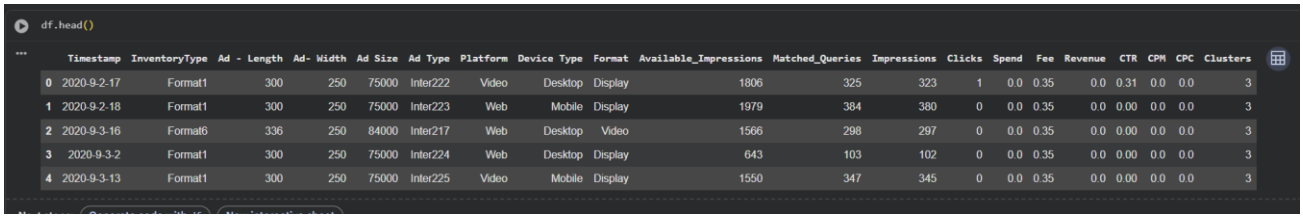
4 clusters scores 0.5109. That is lower than 2, 3 and 8, and we want to be straightforward about that rather than dress it up. What 4 has going for it is that three separate lines of evidence point at it: the elbow sits there, the dendrogram cuts there, and the 4 groups that come out map onto 4 genuinely different kinds of ad inventory, which we show in the next section. 8 clusters scores marginally better at 0.5448 but produces segments too thin to act on.

So, the decision is 4, taken on the evidence together rather than on the silhouette alone. The assignment explicitly asks us to combine the score with domain understanding, and this is where that judgement gets used.

The four clusters

Cluster numbering runs from 1 to 4. Averages are per ad.

Cluster	Ads	Impressions	Clicks	CTR %	CPM	CPC
Cluster 1	6,663	34,74,171	7,252	0.24	1.75	0.78
Cluster 2	3,400	57,727	8,796	21.14	18.68	0.10
Cluster 3	6,528	6,32,049	2,724	0.33	1.08	0.19
Cluster 4	9,266	1,25,901	16,220	12.31	11.63	0.10



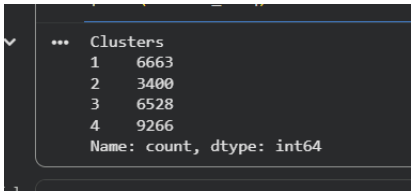
```
df.head()
```

	Timestamp	InventoryType	Ad - Length	Ad- Width	Ad Size	Ad Type	Platform	Device Type	Format	Available_Impressions	Matched_Queries	Impressions	Clicks	Spend	Fee	Revenue	CTR	CPM	CPC	Clusters
0	2020-9-2-17	Format1	300	250	75000	Inter222	Video	Desktop	Display	1806	325	323	1	0.0	0.35	0.0	0.31	0.0	0.0	3
1	2020-9-2-18	Format1	300	250	75000	Inter223	Web	Mobile	Display	1979	384	380	0	0.0	0.35	0.0	0.00	0.0	0.0	3
2	2020-9-3-16	Format6	336	250	84000	Inter217	Web	Desktop	Video	1566	298	297	0	0.0	0.35	0.0	0.00	0.0	0.0	3
3	2020-9-3-2	Format1	300	250	75000	Inter224	Web	Desktop	Display	643	103	102	0	0.0	0.35	0.0	0.00	0.0	0.0	3
4	2020-9-3-13	Format1	300	250	75000	Inter225	Video	Mobile	Display	1550	347	345	0	0.0	0.35	0.0	0.00	0.0	0.0	3

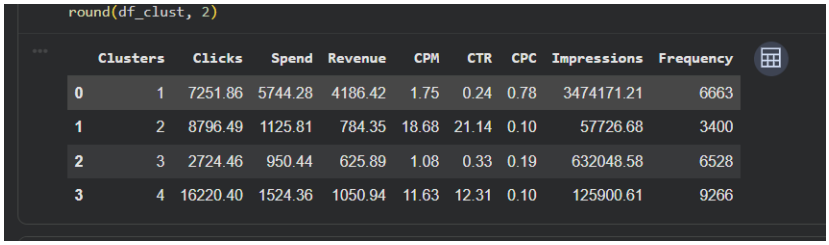
And the share of the total business each one carries:

Cluster	Share of ads	Impressions	Clicks	Spend	Revenue
Cluster 1	25.8%	80.8%	19.6%	61.3%	62.8%
Cluster 2	13.1%	0.7%	12.1%	6.1%	6.0%
Cluster 3	25.2%	14.4%	7.2%	9.9%	9.2%
Cluster 4	35.8%	4.1%	61.0%	22.6%	21.9%

The 2 share columns worth staring at are impressions and clicks. Cluster 1 absorbs 80.8% of every impression the company serves and returns 19.6% of the clicks. Cluster 4 does the opposite, taking 4.1% of impressions and producing 61% of all clicks. That single contrast is the most useful thing this segmentation produced.



```
Clusters
1    6663
2    3400
3    6528
4    9266
Name: count, dtype: int64
```



```
round(df_clust, 2)
```

	Clusters	Clicks	Spend	Revenue	CPM	CTR	CPC	Impressions	Frequency
0	1	7251.86	5744.28	4186.42	1.75	0.24	0.78	3474171.21	6663
1	2	8796.49	1125.81	784.35	18.68	21.14	0.10	57726.68	3400
2	3	2724.46	950.44	625.89	1.08	0.33	0.19	632048.58	6528
3	4	16220.40	1524.36	1050.94	11.63	12.31	0.10	125900.61	9266

```
# Profiling using the SUM of every metric, which shows the total business contribution
# of each cluster rather than the per ad average.
df_clust_sum = df1.groupby('Clusters').sum().reset_index()
round(df_clust_sum, 0)
```

	Clusters	Clicks	Spend	Revenue	CPM	CTR	CPC	Impressions
0	1	48319116	38274117.0	27894132.0	11668.0	1585.0	5217.0	23148402751
1	2	29908078	3827742.0	2666801.0	63513.0	71874.0	337.0	196270703
2	3	17785304	6204460.0	4085820.0	7041.0	2130.0	1251.0	4126013154
3	4	150298217	14124712.0	9738053.0	107718.0	114102.0	910.0	1166595064

```
# Profiling by DEVICE TYPE, as asked in the assignment.
device_profile = df.groupby(['Clusters', 'Device Type'])[
    ['Clicks', 'Spend', 'Revenue', 'CPM', 'CTR', 'CPC']].mean()
round(device_profile, 2)
```

Clusters	Device Type	Clicks	Spend	Revenue	CPM	CTR	CPC
1	Desktop	7333.68	5783.54	4217.86	1.75	0.24	0.78
	Mobile	7206.71	5722.62	4169.08	1.75	0.24	0.79
2	Desktop	9229.77	1159.61	809.18	18.53	21.18	0.10
	Mobile	8546.82	1106.33	770.05	18.77	21.12	0.10
3	Desktop	2794.43	984.91	649.35	1.08	0.33	0.19
	Mobile	2685.79	931.38	612.92	1.08	0.32	0.19
4	Desktop	16280.40	1520.19	1047.45	11.63	12.30	0.10
	Mobile	16187.21	1526.67	1052.88	11.62	12.32	0.10

```
# Count of records by cluster and device type.
print(pd.crosstab(df['Clusters'], df['Device Type']))
```

```
Device Type  Desktop  Mobile
Clusters
1               2369    4294
2               1243    2157
3               2324    4204
4               3300    5966
```

```
# Count of records by cluster and platform.
print(pd.crosstab(df['Clusters'], df['Platform']))
```

```
Platform   App  Video   Web
Clusters
1         1439   2850  2374
2          716   1457  1227
3         1403   2809  2316
4         1986   3961  3319
```

```
# Count of records by cluster and format.
print(pd.crosstab(df['Clusters'], df['Format']))

# FINDING : Desktop and Mobile split roughly 36:64 inside every single cluster, and the
# same is true for Platform and Format. This means the clusters are being driven by how
# the ad performs and what it costs, not by the device it was served on.
```

```
Format    Display  Video
Clusters
1            3341   3322
2            1717   1683
3            3250   3278
4            4621   4645
```

Cluster 1: bulk reach, expensive clicks

6,663 ads, a quarter of the file. These are the giants. An average ad here collects 34.7 lakh impressions, roughly 60 times what a Cluster 2 ad sees. The cost/thousand impressions is low at 1.75, so buying the eyeballs is cheap.

The problem is that almost nobody clicks. CTR is 0.24%, the lowest of the 4. Because spend is high and clicks are scarce, the cost per click lands at 0.78, which is 4x to 8x what the other clusters pay. On a blended basis the cluster pays 0.79 per click against 0.09 in Cluster 4.

This is where the money is. Cluster 1 carries 62.8% of revenue and 61.3% of spend. It is the commercial backbone and also the least click efficient inventory the company owns.

Cluster 2: small, expensive, and people actually click

3,400 ads, the smallest group. Average impressions are 57,727, which is tiny next to Cluster 1. CPM is the highest in the file at 18.68, so these placements cost a lot per thousand views.

They earn it. CTR is 21.14%, by a distance the highest, and the cost/click drops to 0.10. Advertisers pay a premium for the slot and get engagement that justifies it. In practice these look like tightly targeted or native placements where the ad matches what the reader was already doing.

The cluster contributes 6% of revenue from 0.7% of impressions, which tells you how concentrated the value is.

Cluster 3: cheap inventory that is not working

6,528 ads. CPM is the lowest in the file at 1.08 and CTR is 0.33%, barely above Cluster 1. Average revenue per ad is 626, the weakest of the four.

This is filler. It is cheap to buy and it behaves like cheap inventory: 14.4% of impressions producing 7.2% of clicks. It is the only cluster where the honest recommendation is to shrink rather than optimise.

Cluster 4: the workhorse

9,266 ads, the largest cluster at 35.8% of the file, and the one that does the heavy lifting on engagement. CTR is 12.31%, CPM is 11.63 and CPC is 0.10. The average ad pulls 16,220 clicks, more than double Cluster 1 despite serving a twenty seventh of the impressions.

61% of every click ads24x7 delivers comes from this cluster, off 4.1% of impressions. It sits between the bulk reach of Cluster 1 and the boutique economics of Cluster 2, and for most advertisers it is probably the right default buy.

What separates them, in one view

The scaled cluster centres make the structure obvious. These are z-scores, so zero is the overall average and the sign tells the direction.

Cluster	CTR	CPM	CPC	Reads as
Cluster 1	-0.87	-0.81	+1.47	Cheap views, costly clicks
Cluster 2	+1.68	+1.64	-0.61	Costly views, cheap clicks
Cluster 3	-0.86	-0.91	-0.32	Cheap everything, low engagement
Cluster 4	+0.61	+0.62	-0.61	Balanced, high engagement

Clusters 1 and 3 both have low CTR and low CPM, and what separates them is CPC. Cluster 1 pays heavily per click while Cluster 3 does not, because Cluster 1 spends far more in absolute terms to chase the same scarce clicks. Clusters 2 and 4 are both high engagements, and the difference there is intensity: Cluster 2 is the extreme version of what Cluster 4 does.

Device type, and a finding we did not expect

The brief asks us to profile the clusters by device type, so we did. The result is a negative finding and we think it is worth reporting plainly rather than dressing up.

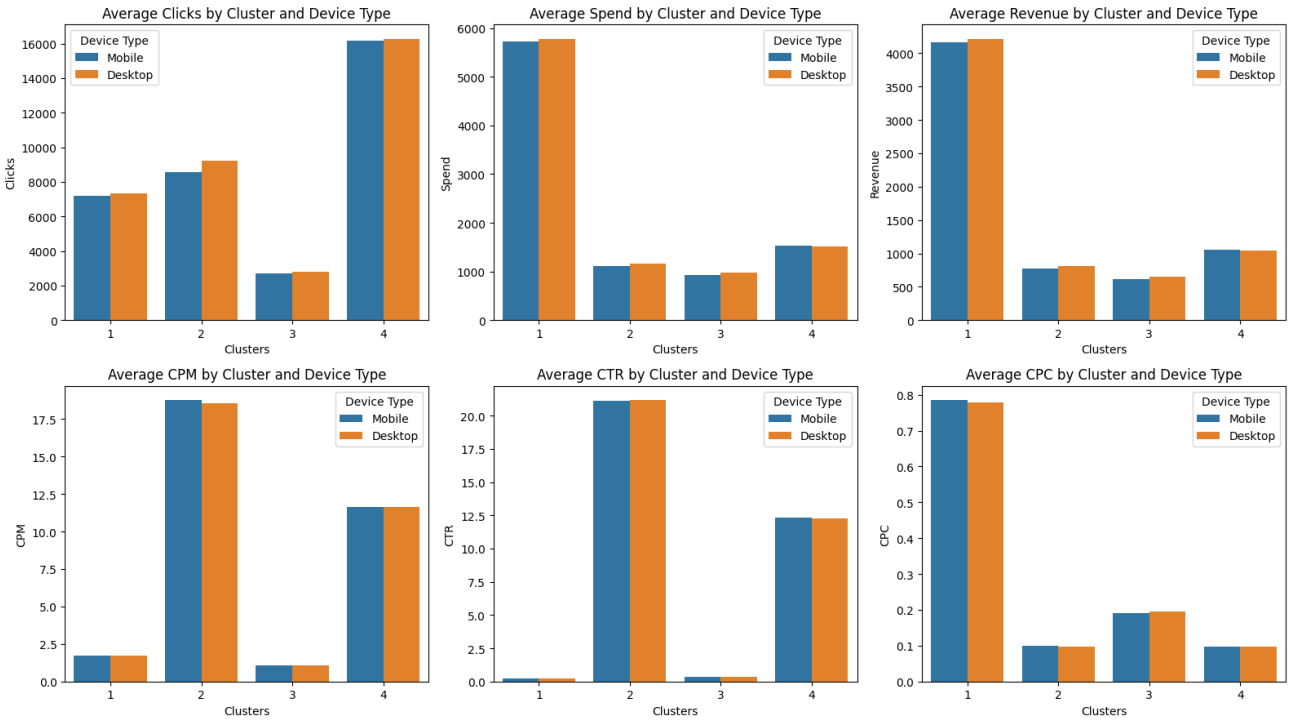
Cluster	Desktop ads	Mobile ads	Mobile share	CTR desktop	CTR mobile
Cluster 1	2,369	4,294	64.4%	0.24	0.24
Cluster 2	1,243	2,157	63.4%	21.18	21.12
Cluster 3	2,324	4,204	64.4%	0.33	0.32
Cluster 4	3,300	5,966	64.4%	12.30	12.32

Mobile accounts for almost exactly 64% of ads inside every cluster, and the performance metrics barely move between desktop and mobile within any of them. The same flatness shows up when we cut by platform and by format.

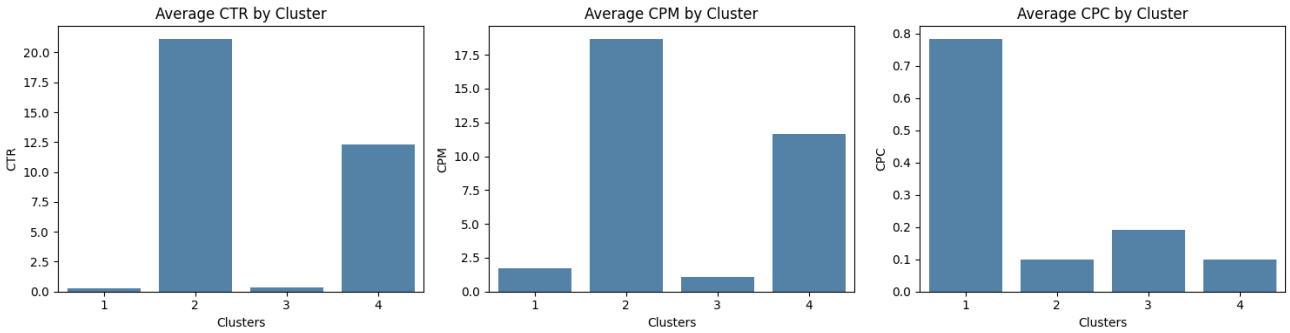
What this means is that the segmentation is driven by how an ad performs and what it costs, not by the device it was served on. Device is not a segmenting variable in this data. That is a useful thing for the marketing team to know, because it says a device led targeting strategy will not by itself move any of these metrics. The lever is placement quality and inventory type, not screen size.

It also means we should resist writing a device story into the recommendations just because the assignment mentions device. The data does not support one.

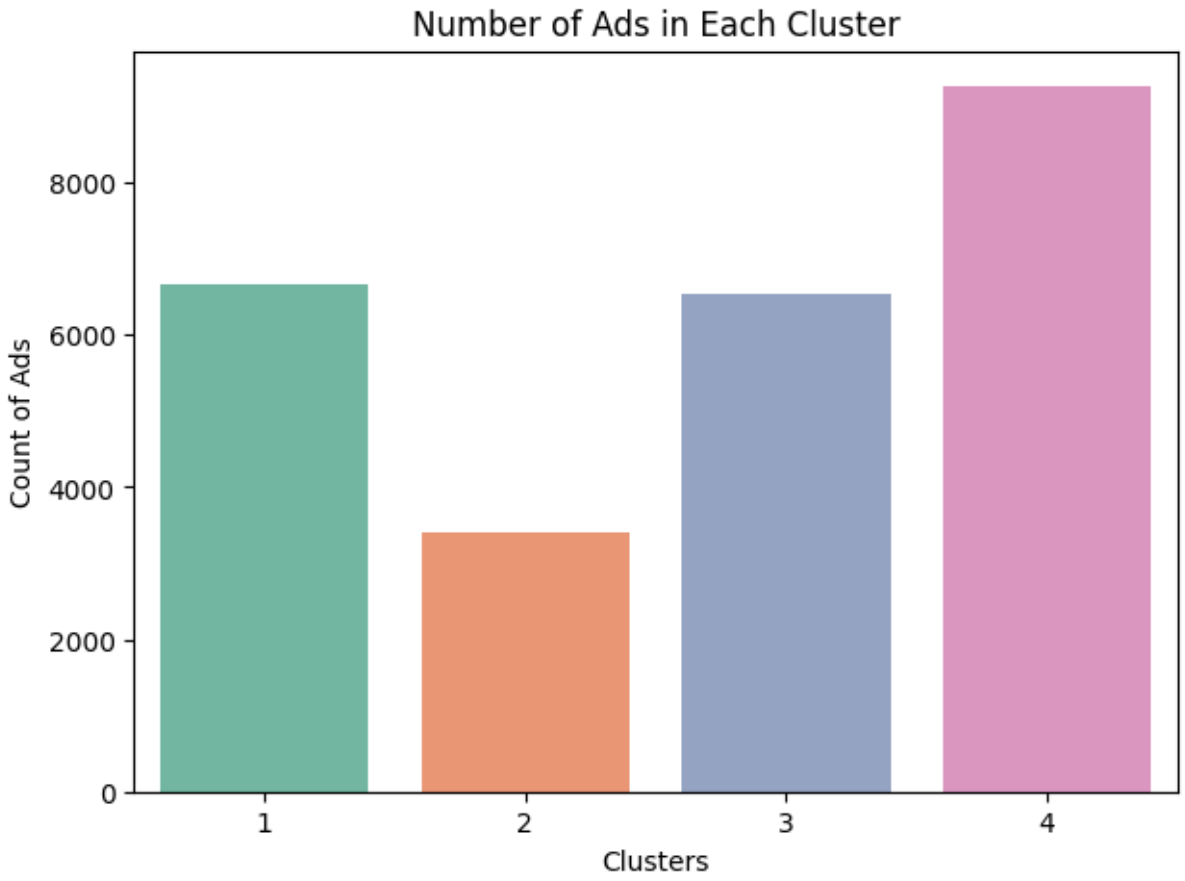
Bar Plots of the key metrics by Cluster and Device Type -



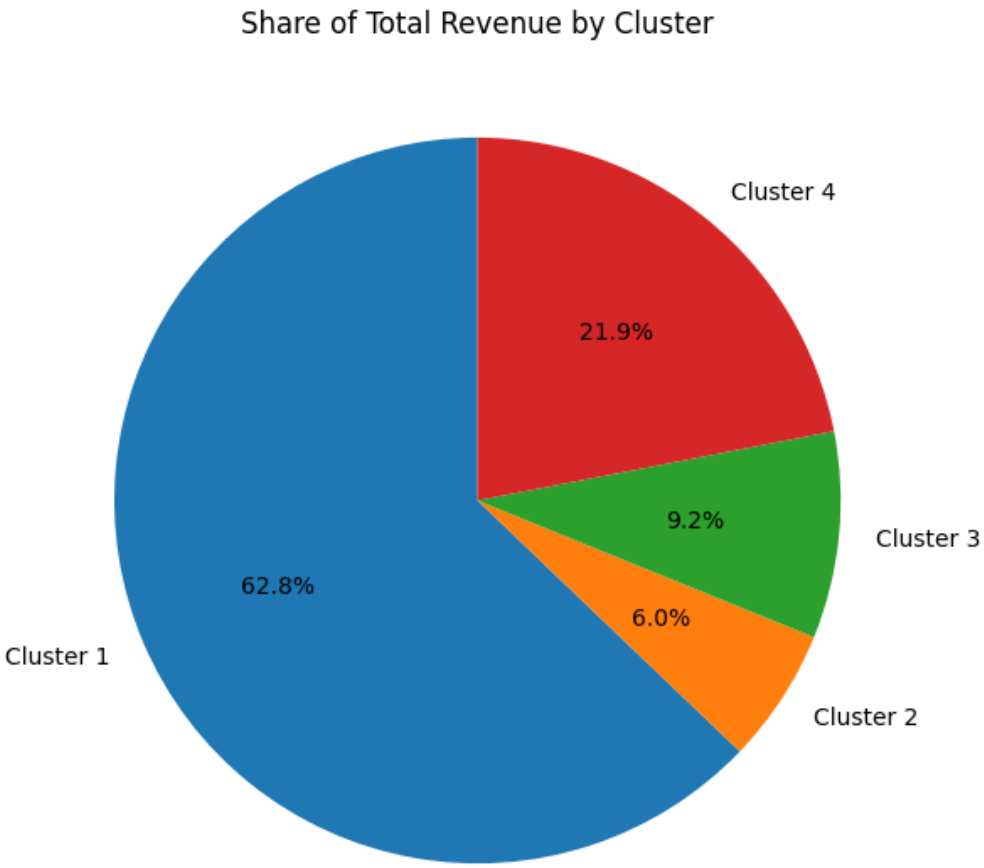
Bar Plot of the Average of Each Clustering Variable -



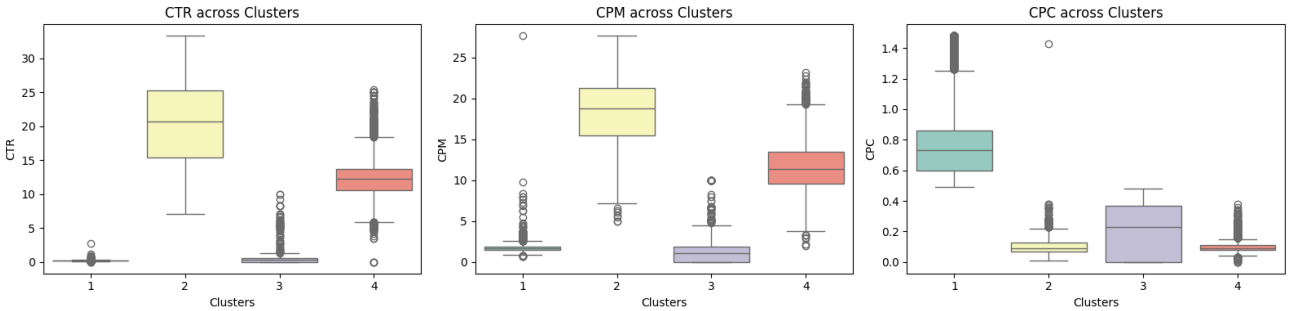
Number of Ads in Each Cluster -



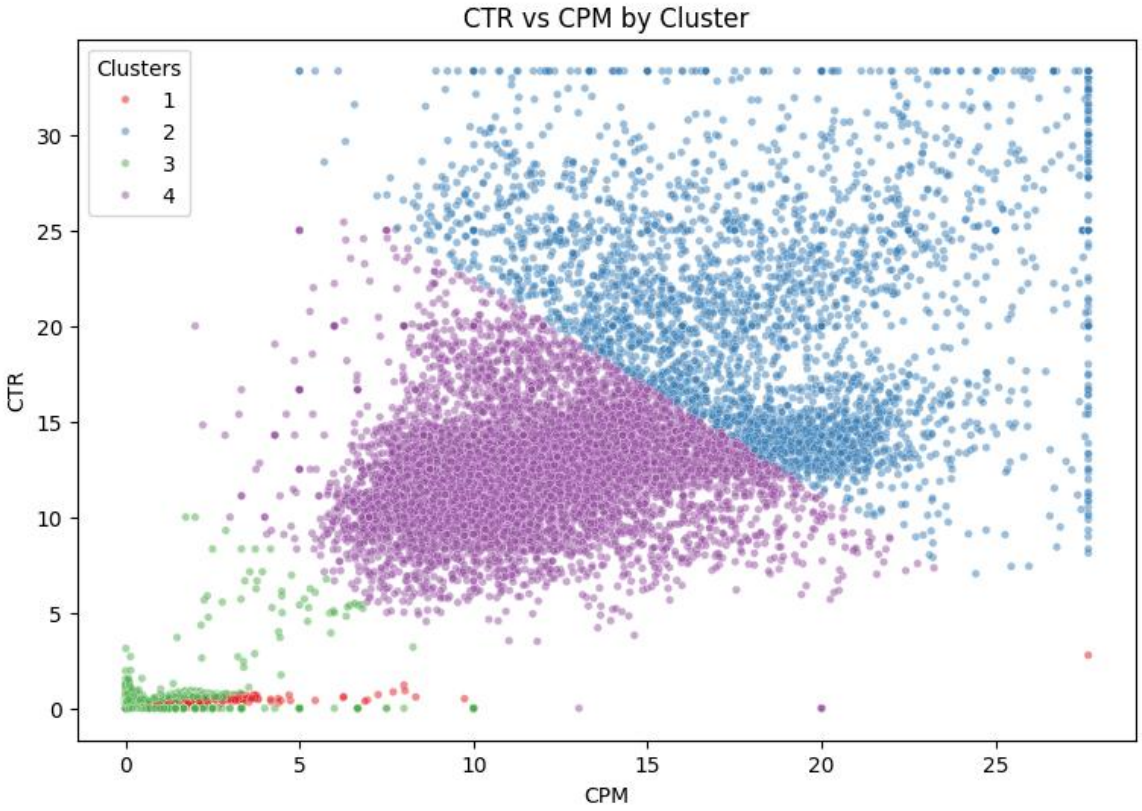
Share of total revenue by cluster -



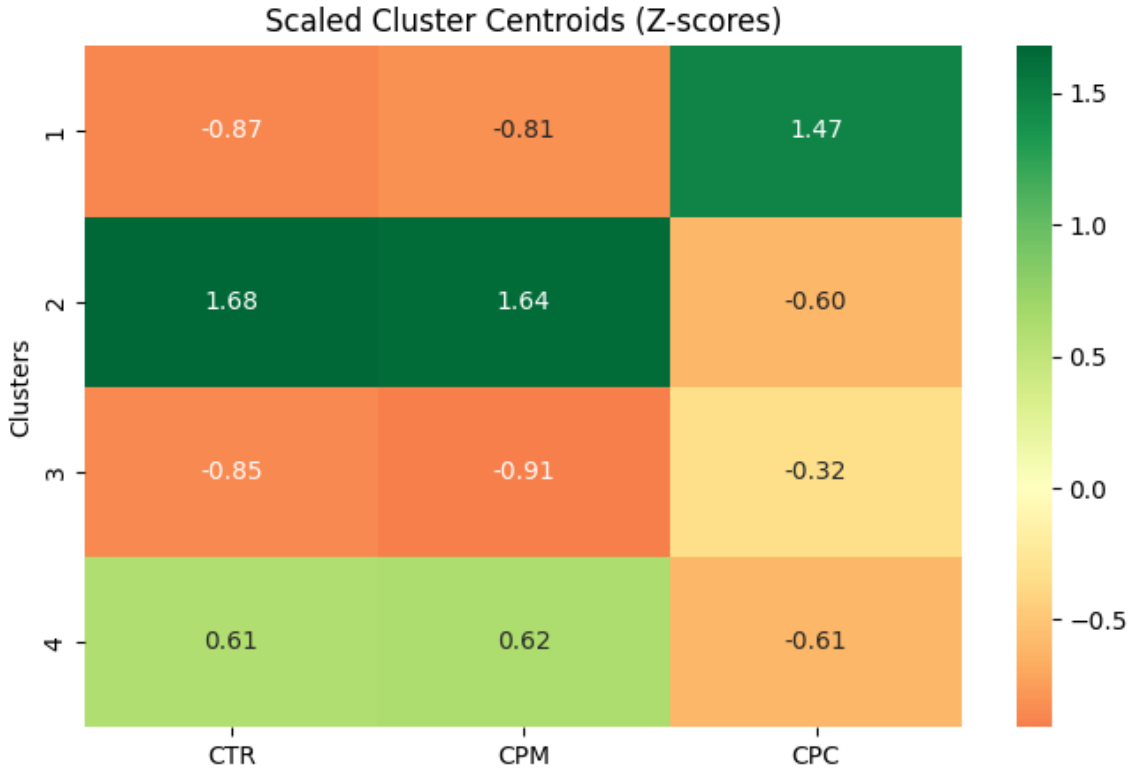
Boxplots of the clustering variables across the clusters -



Scatter plot of CTR against CPM coloured by cluster -



Heatmap of the scaled cluster centroids -



Hierarchical or K-Means

We ran both and report K-Means as the final answer.

Consideration	Hierarchical (Ward)	K-Means
Full data set	Not feasible, needs 334M pairwise distances	Runs on all 25,857 rows
Speed	Slow, roughly n squared	Fits in under a second
Choosing the count	Read off the dendrogram after building it	Must be fixed in advance
Output	Tree, good for seeing structure	Centroids, good for profiling
Reproducible	Yes, deterministic	Yes, with a fixed random state

The 2 methods were compared on the same 2,000 records. The Adjusted Rand Index came out at 0.6664, and once the arbitrary cluster numbering is matched up they place 83.4% of records in the same group. So, they agree on the broad structure without agreeing on every record, and the disagreement sits mostly on ads near the cluster boundaries, which is normal.

That agreement matters for the argument. It says the 4 cluster structure is a property of the data rather than something one algorithm invented. Having established that, we go with K-Means because it runs on the complete file while hierarchical clustering had to be sampled, and because centroids translate directly into the profile tables above whereas a dendrogram does not.

```
# The cluster numbers themselves are arbitrary in both methods, so comparing the sizes
# side by side tells us nothing. We have to cross tabulate them instead.
print(pd.crosstab(pd.Series(clusters, name='Hierarchical'),
                  pd.Series(sample_kmeans, name='K-Means')))
```

```
K-Means         1    2    3    4
Hierarchical
1               0  143    0   29
2               0  116    7  714
3             328    0    0    0
4             181    0  482    0
```

```
# FINDING : The Adjusted Rand Index is 0.67 and once the arbitrary cluster numbers are
# matched up, the two methods put 83.4% of the sampled records in the same group. So they
# agree on the broad shape of the data but not on every record, and the disagreement sits
# mostly on the records lying near the cluster boundaries. That level of agreement is
# enough to say the 4 cluster structure is real and not an artefact of one algorithm.

# We report K-Means as the final answer because it runs on all 25857 records, whereas
# hierarchical clustering had to be run on a sample of 2000 for memory reasons.
```

```
Adjusted Rand Index : 0.6664
```

Business recommendations

Cluster 1

- Sell this as a reach product and price it on CPM, not on clicks. The click economics are poor and any advertiser buying it for performance will be disappointed.
- Attach frequency capping. At 34.7 lakh impressions per ad, a meaningful share of those views are repeat exposures to the same person, and that is likely part of why CTR sits at 0.24%.
- Protect the fill rate here before anything else. This cluster is 62.8% of revenue, so a few points of unsold inventory costs more than any optimisation elsewhere would earn.
- Run a test that moves a slice of this budget into Cluster 4 placements and compare cost per click. If the pattern holds, there is a real efficiency argument to take to advertisers.

Cluster 2

- This is the premium tier. Price it accordingly and defend the CPM floor, because the 21% click through rate is what the advertiser is actually buying.
- Find out what makes it work. This is the single highest value analytics question in the file. If the driver is contextual matching or a particular slot position, some of it may transfer to Cluster 4 inventory.
- Be careful about scaling it. Only 0.7% of impressions sit here, and high engagement often depends on scarcity and tight targeting. Volume could easily destroy the thing that makes it valuable.
- Package it as a premium add on alongside a Cluster 1 reach buy rather than selling it standalone.

Cluster 3

- Reduce exposure to this inventory. It takes 14.4% of impressions and returns 7.2% of clicks, and it has the lowest revenue per ad in the file.
- Before cutting, check whether the low CPM reflects genuinely poor placements or simply unsold slots being cleared at a discount. The remedy differs completely depending on which it is.
- Whatever survives the review should be repriced or moved into the Cluster 4 mix rather than sold on its own.

Cluster 4

- Make this the default recommendation for most advertisers. It delivers 61% of clicks at a blended cost of 0.09 per click, which is the best combination of scale and efficiency in the file.
- Grow it. This is the clearest expansion opportunity, and it currently uses only 4.1% of impressions, so there is room.
- Use it as the testing ground for creative work. It is large enough to give clean reads and stable enough that a test will not distort the quarter.
- Watch CPC drift as volume grows. The economics here depend on cheap clicks, and that is the first thing that tends to erode when you scale.

Summary of learnings

On the method

Imputation should use whatever the domain gives you. Filling 6,465 missing CTR values with a column mean would have been fast and would have quietly destroyed the variable we were about to cluster on. The formula was sitting right there in three raw columns.

Check the units before trusting a column. The CTR scale mismatch was not visible in the summary statistics and only turned up when we recomputed a value by hand and compared it against what was stored. Nothing downstream would have thrown an error, we would just have been clustering on a column with two different meanings inside it.

Statistical criteria disagree, and that is normal. The silhouette said 2 clusters, the elbow said 4 and the dendrogram said 3 or 4. Waiting for them to line up is not an option, and picking whichever one supports the answer you already wanted is not analysis. We took 4 because 3 lines of evidence converged there and because the resulting segments were things a marketing team could actually price and sell.

Scale before running a distance based algorithm, and know why. It is not a formality. CPM would have dominated the distance calculation purely on the size of its numbers, and the clusters would have been a CPM ranking wearing a disguise.

Cross checking with a second method is worth the effort. Running hierarchical clustering on a sample and getting 83.4% agreement with K-Means gave us something to point at when justifying the 4 cluster choice, which a single silhouette score of 0.51 would not have.

On the business

The 4 segments are genuinely different and they split along the axis of scale versus engagement. Cluster 1 buys attention in bulk and cheaply, then pays dearly for each click. Cluster 4 does the reverse. Clusters 2 and 3 sit at the extremes of that same trade off.

The most actionable number in the whole analysis is the impressions to clicks contrast. Cluster 1 takes 80.8% of impressions and returns 19.6% of clicks. Cluster 4 takes 4.1% and returns 61%. Whether that gap can be closed, and how far, is the question we would take into the next piece of work.

Device type does not segment this data. We went looking for a mobile versus desktop story because the brief pointed at one, and the numbers came back flat across all 4 clusters. Reporting that honestly is more useful than manufacturing a pattern out of a 0.02 percentage point difference in CTR.

Revenue turned out not to be a performance metric at all, just spend net of fee. It is a reminder to check what a column actually contains before building an argument on it.